

MASEMBO PARKING SYSTEM

TECHNICAL DESIGN REPORT

System Modules, Architecture, and Pseudocode

Prepared for: Masembo Parking Management
Prepared by: Masembo Systems Development Team

Document Version: 1.0

September 2026

Table of Contents

1. Introduction

- 1.1 Purpose of this Document
- 1.2 Project Overview
- 1.3 Scope

2. System Architecture Overview

- 2.1 Technology Stack
- 2.2 Component Overview

3. Core Data Structures and Algorithms

- 3.1 Array — Slot Inventory
- 3.2 Queue — Available Slot Allocation
- 3.3 Hash Map — Active Session Lookup
- 3.4 Complexity Summary

4. Module 1: Vehicle Entry Module

5. Module 2: Vehicle Exit and Fee Calculation Module

6. Module 3: Payment Processing Module

- 6.1 Cash Payment Sub-module
- 6.2 Lipa Na M-Pesa Online (STK Push) Sub-module

7. Module 4: Barrier Control Module

8. Module 5: Staff Authentication Module

9. Database Design

10. Testing and Validation

11. Security Considerations

12. Limitations and Future Enhancements

13. Conclusion

Appendix A — Consolidated Pseudocode Index

Appendix B — Glossary of Terms

Appendix C — Sample End-to-End Walkthrough

1. Introduction

1.1 Purpose of this Document

This report documents the design and internal workings of the Masembo Parking System, a Flask-based smart-gate application that manages vehicle entry, slot allocation, fee calculation, and exit payment for a paid parking facility. It is intended for developers, technical reviewers, and system maintainers who need a clear description of each functional module together with the pseudocode that governs its behaviour.

The document covers the system as currently implemented, including the recently added Lipa Na M-Pesa Online payment integration and the shared-password staff login that gates entry and payment actions.

1.2 Project Overview

Masembo Parking System replaces manual ticketing with a lightweight web application an attendant operates from a browser at the gate. A driver's number plate is recorded on arrival, the system tracks how long the vehicle has been parked, calculates a fee on exit according to tiered pricing, and opens a (simulated) barrier once payment is confirmed — either in cash or via an M-Pesa STK push sent to the driver's phone.

The system is built around a small set of cooperating modules, each responsible for one stage of the vehicle's journey through the car park: Entry, Exit & Fee Calculation, Payment Processing, Barrier Control, and Staff Authentication. These are described in detail in Sections 4 through 8.

1.3 Scope

This report covers the web application layer (Flask routes), the in-memory and persisted data model (SQLite), the core allocation algorithms, and the third-party payment integration. It does not cover physical barrier hardware, camera-based plate recognition, or network infrastructure, all of which are outside the current implementation and noted as future work in Section 12.

2. System Architecture Overview

The system follows a simple three-layer structure common to small Flask applications: a presentation layer of server-rendered HTML templates, an application layer that holds business logic and in-memory state, and a persistence layer backed by SQLite.

2.1 Technology Stack

Layer	Technology	Responsibility
Web framework	Flask 3.0	Routing, request handling, session cookies
Templating	Jinja2	Server-rendered HTML for dashboard, checkout, receipts
Database	SQLite 3	Durable storage of slots and parking sessions
Payment gateway	Safaricom Daraja API	Lipa Na M-Pesa Online (STK Push)

Layer	Technology	Responsibility
HTTP client	requests	Calls to the Daraja OAuth and STK push endpoints
Front end	Vanilla CSS/JS	Slot grid, waiting-screen polling, styling

2.2 Component Overview

Four Python modules divide responsibility cleanly:

- `app.py` — Flask routes; wires HTTP requests to the business logic and renders templates.
- `algorithms.py` — the `SlotManager` class; holds the slot array, the available-slot queue, and the active-session hash map, and implements entry, checkout, and M-Pesa state transitions.
- `database.py` — SQLite connection handling and schema initialisation, including safe migrations for new columns.
- `mpesa.py` — a thin client for Safaricom's Daraja API: OAuth token retrieval, phone-number normalisation, and the STK push request itself.

Figure 2.1 below summarises the request flow for a typical exit-and-pay transaction.

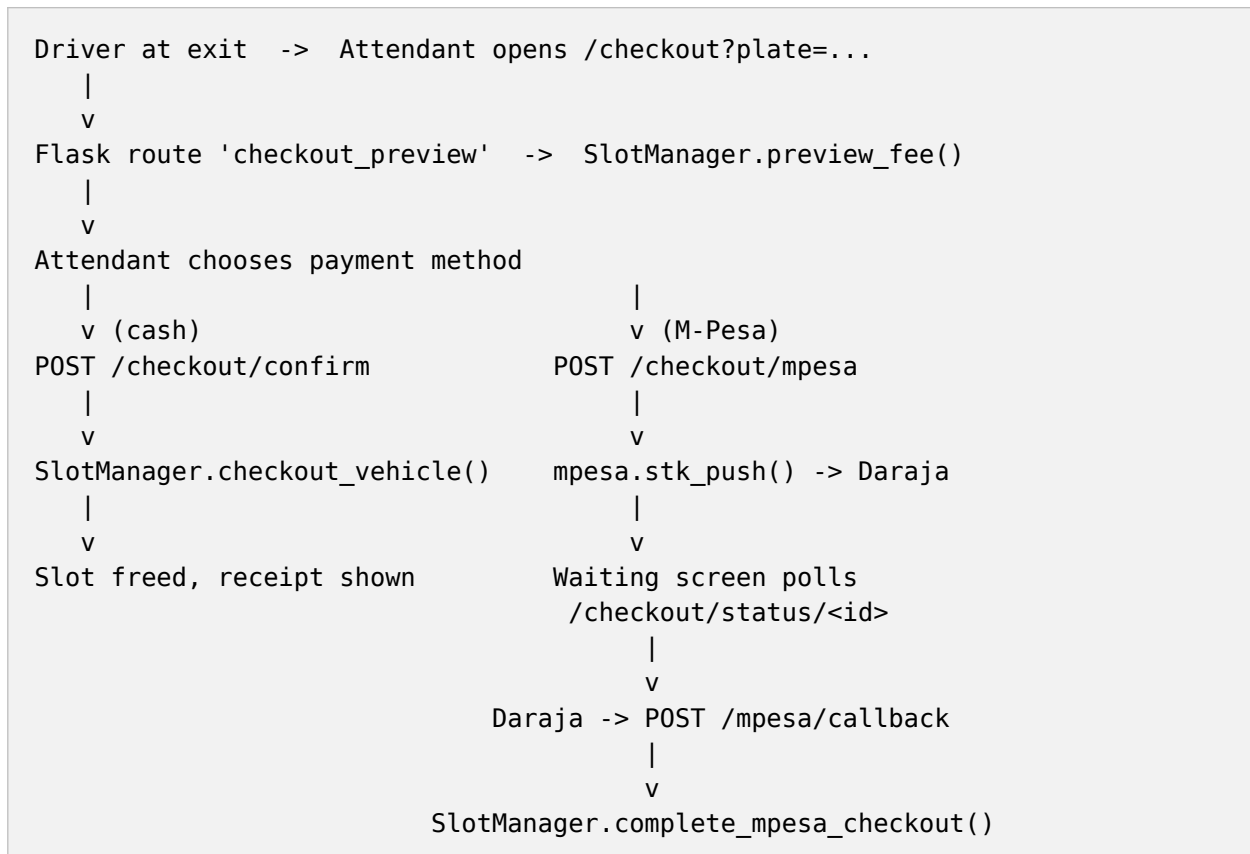


Figure 2.1 — Request flow for a checkout transaction, cash versus M-Pesa.

3. Core Data Structures and Algorithms

The SlotManager class at the heart of algorithms.py is deliberately built from three classic data structures, chosen so that every hot-path operation — checking a slot, allocating the next free one, and finding an active session — runs in constant or near-constant time.

3.1 Array — Slot Inventory

All parking slots are held in an ordered Python list (self.slots), indexed identically to their position in the database. This gives $O(1)$ access by position and a natural iteration order for rendering the dashboard's slot grid.

3.2 Queue — Available Slot Allocation

Free slots are tracked in a FIFO queue (collections.deque) of slot numbers. When a vehicle arrives, the manager pops from the left of the queue to allocate the longest-waiting free slot; when a vehicle leaves, its slot number is pushed to the right of the queue. This guarantees fair, predictable rotation of slots rather than always reusing the same low-numbered ones.

```
FUNCTION allocate_slot():
    IF available_queue IS EMPTY:
        RETURN None // car park is full
    slot_number <- available_queue.pop_left()
    RETURN slot_number

FUNCTION release_slot(slot_number):
    available_queue.push_right(slot_number)
```

3.3 Hash Map — Active Session Lookup

Active parking sessions are indexed in a Python dict (self.active_sessions) keyed by plate number, so that looking up 'is this plate currently parked, and in which slot, since when' is an $O(1)$ operation rather than a linear scan. A second hash map, pending_mpesa, maps a Safaricom CheckoutRequestID back to a plate number so that an asynchronous payment callback can be resolved instantly to the right session.

3.4 Complexity Summary

Operation	Structure Used	Time Complexity
Record vehicle entry	Queue + Hash Map	$O(1)$
Look up active session by plate	Hash Map	$O(1)$
Free a slot on exit	Queue	$O(1)$
Render full slot grid	Array	$O(n)$
Resolve M-Pesa callback to plate	Hash Map	$O(1)$

Operation	Structure Used	Time Complexity
Fee calculation	Arithmetic on timestamps	O(1)

4. Module 1: Vehicle Entry Module

The Entry module records a vehicle's arrival: it validates the plate number, allocates the next available slot from the queue, timestamps the entry, and persists a new active session row. It is reachable only to a logged-in attendant (see Section 8).

Responsibilities

- Normalise and validate the submitted plate number.
- Reject entry if the car park is full (available queue empty).
- Reject entry if the same plate already has an active session.
- Allocate a slot, mark it occupied, and open a new session record.

Pseudocode

```

MODULE VehicleEntry

FUNCTION record_entry(plate_number):
    plate_number <- TRIM(plate_number).TO_UPPER()
    IF plate_number IS EMPTY:
        RETURN FAIL('Enter a plate number.')

    IF plate_number IN active_sessions:
        RETURN FAIL('This plate already has an active session.')

    slot_number <- allocate_slot()
    IF slot_number IS None:
        RETURN FAIL('Car park is full – no available slots.')

    entry_time <- NOW()
    session_id <- DATABASE.insert_session(
        plate_number, slot_number, entry_time, status='active')
    slot.status <- 'occupied'
    slot.plate_number <- plate_number
    active_sessions[plate_number] <- {
        session_id, slot_id, entry_time }

    RETURN SUCCESS('Vehicle parked in slot ' + slot_number)

```

On success, the dashboard immediately re-renders with the newly occupied slot shown in red, and the available/occupied counters update to reflect the new totals.

5. Module 2: Vehicle Exit and Fee Calculation Module

Before any payment is taken, the attendant looks up a plate to preview the fee owed. The fee is calculated from a tiered pricing table based on the number of minutes the vehicle has been parked, with a free allowance for very short stays.

Fee Tiers

Duration Parked	Fee (Kshs.)
0 – 30 minutes	0 (free)
31 – 60 minutes	50
1 – 3 hours	100
3 – 6 hours	200
Over 6 hours	300 + 50 per additional hour

Pseudocode — Fee Preview

```

FUNCTION preview_fee(plate_number):
    session <- active_sessions.get(plate_number)
    IF session IS None:
        RETURN None    // no active session for this plate

    minutes <- (NOW() - session.entry_time) IN minutes
    fee <- calculate_fee(minutes)
    RETURN { plate_number, slot_number, entry_time, minutes, fee }

FUNCTION calculate_fee(minutes):
    IF minutes <= 30:           RETURN 0
    ELSE IF minutes <= 60:      RETURN 50
    ELSE IF minutes <= 180:     RETURN 100
    ELSE IF minutes <= 360:     RETURN 200
    ELSE:
        extra_hours <- CEIL((minutes - 360) / 60)
        RETURN 300 + (extra_hours * 50)

```

Pseudocode — Finalising a Cash Checkout

Once the attendant confirms cash has been received, the session is closed, the slot is freed back into the allocation queue, and the payment method is explicitly recorded as 'cash' — overwriting any stale tag left behind by an earlier, abandoned M-Pesa attempt on the same session (see Section 6.2).

```

FUNCTION checkout_vehicle(plate_number):
  session <- active_sessions.get(plate_number)
  IF session IS None:
    RETURN FAIL('No active session found for this plate.')

  exit_time <- NOW()
  fee <- calculate_fee(minutes_between(session.entry_time, exit_time))

  DATABASE.update_session(session.id,
    exit_time=exit_time, fee=fee, status='completed',
    payment_method='cash', payment_status=NULL, mpesa_receipt=NULL)
  DATABASE.update_slot(session.slot_id, status='available')

  release_slot(session.slot_number)
  REMOVE active_sessions[plate_number]
  RETURN SUCCESS(receipt_details, barrier='OPEN')

```

6. Module 3: Payment Processing Module

A non-zero fee offers the driver two payment routes: cash, confirmed directly by the attendant, or Lipa Na M-Pesa Online, which sends a payment prompt (an 'STK push') straight to the driver's phone. A fee of zero (a free, short stay) bypasses both and completes the checkout immediately, without ever being tagged as an M-Pesa attempt.

6.1 Cash Payment Sub-module

This path is described fully in Section 5, under 'Finalising a Cash Checkout'. It is the simpler of the two payment routes because it completes synchronously within a single request — there is no external party to wait on.

6.2 Lipa Na M-Pesa Online (STK Push) Sub-module

The M-Pesa path is asynchronous: the app cannot complete the checkout the moment the driver clicks 'pay', because the actual payment happens later, on the driver's own phone, and Safaricom reports the result back on its own schedule via a webhook callback. The sub-module is therefore split into three stages.

Stage 1 — Initiating the STK Push

```

FUNCTION checkout_mpesa(plate_number, phone_number):
  IF phone_number IS EMPTY:
    RETURN FAIL('Enter the phone number for the prompt.')

  preview <- preview_fee(plate_number)
  IF preview IS None:
    RETURN FAIL('No active session found.')

  IF preview.fee <= 0:
    RETURN checkout_vehicle(plate_number)    // free exit, skip M-Pesa

  session_id <- initiate_mpesa_checkout(plate_number, phone_number)
  // marks session payment_method='mpesa', payment_status='pending'

  (ok, result) <- mpesa.stk_push(
    phone_number, preview.fee, account_reference=plate_number)
  IF NOT ok:
    RETURN FAIL('Could not send the M-Pesa prompt: ' + result)

  checkout_request_id <- result
  attach_checkout_request_id(session_id, plate_number, checkout_request_id)
  RETURN SHOW_WAITING_SCREEN(session_id)

```

Stage 2 — Safaricom's Callback

Safaricom posts the outcome of the STK push to a public HTTPS endpoint configured on the Daraja app. Only Safaricom's servers call this route; it is deliberately left outside the staff-login requirement so the webhook is never blocked by an authentication wall.

```

FUNCTION mpesa_callback(request_body):
  checkout_request_id <- request_body.Body.stkCallback.CheckoutRequestID
  result_code          <- request_body.Body.stkCallback.ResultCode

  IF result_code == 0:
    mpesa_receipt <- EXTRACT('MpesaReceiptNumber', request_body)
    complete_mpesa_checkout(checkout_request_id, mpesa_receipt)
  ELSE:
    // cancelled, timed out, or insufficient funds
    mark_mpesa_failed(checkout_request_id)

  RETURN ACK(ResultCode=0, ResultDesc='Accepted')

FUNCTION complete_mpesa_checkout(checkout_request_id, mpesa_receipt):
  plate_number <- pending_mpesa.pop(checkout_request_id)
  session <- active_sessions.get(plate_number)
  IF session IS None: RETURN FALSE

  fee <- calculate_fee(minutes_between(session.entry_time, NOW()))
  DATABASE.update_session(session.id, exit_time=NOW(), fee=fee,
    status='completed', payment_status='paid',
    mpesa_receipt=mpesa_receipt)
  DATABASE.update_slot(session.slot_id, status='available')
  release_slot(session.slot_number)
  REMOVE active_sessions[plate_number]
  RETURN TRUE

```

Stage 3 — The Waiting Screen and Status Polling

While waiting for the callback, the driver's browser shows a spinner and polls a lightweight JSON status endpoint every three seconds, for up to two minutes, before giving up and offering a retry.

```

FUNCTION checkout_status(session_id):
  row <- DATABASE.get_session(session_id)
  IF row IS None:
    RETURN { status: 'error' }
  IF row.status == 'completed' AND row.payment_status == 'paid':
    RETURN { status: 'paid', redirect: '/receipt/' + session_id }
  IF row.payment_status == 'failed':
    RETURN { status: 'failed' }
  RETURN { status: 'pending' }

```

7. Module 4: Barrier Control Module

The barrier module is intentionally the simplest in the system: in the current implementation it is simulated in software, returning the string 'OPEN' as part of the receipt data once payment has been

confirmed, by either payment route. On real hardware, this function would instead send a signal — over a relay, GPIO pin, or serial connection — to the physical boom gate controller.

Pseudocode

```
FUNCTION open_barrier(session_result):  
    IF session_result.payment_confirmed == TRUE:  
        // Hardware integration point:  
        // SEND_SIGNAL(barrier_controller, 'OPEN')  
        session_result.barrier <- 'OPEN'  
    RETURN session_result
```

Because both the cash and M-Pesa payment paths converge on the same checkout-completion logic (checkout_vehicle and complete_mpesa_checkout respectively), the barrier is only ever marked open after the database has already recorded the session as completed and paid — there is no path through the code that opens the barrier before payment is confirmed.

8. Module 5: Staff Authentication Module

To prevent an unattended browser tab from being used to record fraudulent entries or accept payment without an attendant present, a lightweight authentication layer gates the state-changing routes. The design intentionally uses a single shared password rather than individual accounts, matching the scale of a single-gate attendant operation.

Design Decisions

- The slot dashboard and fee-preview lookup remain public, so a driver can check availability or their own fee without needing a login.
- Recording an entry, confirming a cash payment, and initiating an M-Pesa push all require an active staff session.
- Safaricom's callback route is intentionally left unauthenticated, since it is called by Safaricom's servers, not a browser.
- The shared password is stored outside source control, in an environment variable, and the session is signed using Flask's secret-key cookie mechanism.

Pseudocode

```

FUNCTION login_required(view_function):
    RETURN WRAPPED_FUNCTION(request):
        IF session.is_staff != TRUE:
            FLASH('Please log in as staff to do that.')
            RETURN REDIRECT('/login', next=request.path)
        RETURN view_function(request)

FUNCTION login(request):
    IF request.method == 'POST':
        IF request.form.password == STAFF_PASSWORD:
            session.is_staff <- TRUE
            RETURN REDIRECT(request.form.next OR '/')
        ELSE:
            FLASH('Incorrect password.')
    RETURN RENDER('login.html')

FUNCTION logout(request):
    REMOVE session.is_staff
    RETURN REDIRECT('/')

// Guarded routes:
// POST /entry @login_required
// POST /checkout/confirm @login_required
// POST /checkout/mpesa @login_required

```

9. Database Design

The system persists two tables in SQLite: slots, one row per physical parking bay, and sessions, one row per vehicle visit — active or completed, cash or M-Pesa.

Table: slots

Column	Type	Notes
id	INTEGER PK	Auto-incrementing identifier
slot_number	TEXT	Display label, e.g. 'S01'
status	TEXT	'available' or 'occupied'

Table: sessions

Column	Type	Notes
id	INTEGER PK	Auto-incrementing identifier
plate_number	TEXT	Vehicle registration

Column	Type	Notes
slot_id	INTEGER FK	References slots.id
entry_time / exit_time	TEXT (ISO 8601)	exit_time is NULL while active
fee	REAL	Computed on checkout
status	TEXT	'active' or 'completed'
phone_number	TEXT	Set only for M-Pesa attempts
payment_method	TEXT	'cash' or 'mpesa'
payment_status	TEXT	'pending' / 'paid' / 'failed' (M-Pesa only)
checkout_request_id	TEXT	Safaricom's STK push identifier
mpesa_receipt	TEXT	e.g. 'SFC1A2B3C4'

The five M-Pesa-related columns were added by an additive migration (ensure_payment_columns) that inspects the existing schema with PRAGMA table_info before issuing any ALTER TABLE statement, so that a database created before this feature existed upgrades safely without losing prior session history.

10. Testing and Validation

Each module was exercised with Flask's built-in test client against a fresh SQLite database, covering both the happy path and the edge cases most likely to produce incorrect payment-method labelling or to leave the system in an inconsistent state (for example, a slot marked occupied with no corresponding active session).

#	Test Case	Expected Result	Outcome
1	Vehicle entry while logged out	Redirected to login, entry blocked	Pass
2	Dashboard visible while logged out	Slot grid renders; entry form hidden	Pass
3	Login with incorrect password	Error flashed, session not granted	Pass
4	Login with correct password	Session granted, UI shows Log Out	Pass
5	Entry while authenticated	Slot allocated, session created	Pass
6	Free exit (0-fee stay) via M-Pesa button	Tagged 'cash', not 'mpesa'	Pass
7	Cash checkout after an abandoned M-Pesa attempt	payment_method reset to 'cash'	Pass
8	STK push with M-Pesa not configured	Graceful error, session stays	Pass

#	Test Case	Expected Result	Outcome
		active	
9	Full M-Pesa happy path (mocked STK push)	Callback frees slot, receipt shows M-Pesa receipt	Pass
10	Status polling before callback arrives	Reports 'pending'	Pass
11	Status polling after successful callback	Reports 'paid' with redirect URL	Pass
12	History page after logout	Still publicly viewable	Pass

All twelve scenarios passed on the current codebase. Tests 6 and 7 were added specifically after an initial implementation defect was caught during manual review: a free exit triggered through the M-Pesa button was briefly mis-tagged as an M-Pesa payment, and a cash payment following an abandoned M-Pesa attempt did not clear the stale 'mpesa' tag. Both were corrected before release.

11. Security Considerations

- Credential handling: Daraja Consumer Key/Secret and the staff password are read from environment variables via a .env file, which is excluded from version control.
- PIN safety: the driver's M-Pesa PIN is entered directly on their own phone and is never transmitted to, or stored by, this application at any point.
- Session cookies: Flask's session is signed with a secret key; the default development key must be replaced with a random value before any real-world deployment.
- Callback endpoint: /mpesa/callback is intentionally public, since Safaricom's servers call it directly — this is standard for webhook endpoints, but production deployments should validate the source IP range or use a shared secret in the callback URL.
- Shared password: a single attendant password is a reasonable starting point for one gate with a small team, but it does not provide an audit trail of which individual performed which action; Section 12 recommends per-attendant accounts as the system scales.
- In-memory state: the pending_mpesa mapping used to resolve callbacks lives in the application process's memory and is lost on restart; see Section 12 for the recommended production hardening.

12. Limitations and Future Enhancements

Current Limitations

- An in-flight STK push is forgotten if the server restarts before Safaricom's callback arrives, since pending_mpesa is an in-memory dictionary rather than a database table.
- Staff authentication uses one shared password with no per-attendant identity or audit trail.
- The barrier and plate capture are both simulated; there is no integration with physical hardware or camera-based recognition yet.

- The system currently supports a single site; there is no notion of multiple branches or car park levels.

Recommended Enhancements

1. Persist pending_mpesa as a database table (or add a periodic 'query STK status' job) so an in-flight payment survives a server restart.
2. Replace the shared password with individual attendant accounts and hashed credentials, enabling a per-user audit trail.
3. Add a level or branch_id column to support multiple car parks from one deployment.
4. Integrate a real barrier controller and, optionally, automatic plate recognition at the entry camera.
5. Add role-based access (attendant vs. manager) so only managers can view financial history and reports.

13. Conclusion

The Masembo Parking System demonstrates that a small, well-structured set of modules — built on an array, a queue, and a hash map — is sufficient to run a real paid parking workflow end to end: recording arrivals, calculating tiered fees, accepting both cash and mobile-money payment, and gating those actions behind a staff login, all while keeping every hot-path lookup at constant time.

The addition of Lipa Na M-Pesa Online required the most careful design of the three payment-related modules, because it is the only one that spans two separate HTTP requests to two separate parties (the attendant's browser and Safaricom's servers) connected only by a shared CheckoutRequestID. The testing summarised in Section 10 confirms that this asynchronous handoff, and the edge cases around it, behave correctly.

The limitations documented in Section 12 are scoped intentionally: none of them block correct operation at the current single-gate scale, and each has a clear, incremental path to being addressed as the system grows.

Appendix A — Consolidated Pseudocode Index

For quick reference, this appendix lists every pseudocode routine defined in this report and the module it belongs to.

Routine	Module	Section
allocate_slot() / release_slot()	Core Data Structures	3.2
record_entry()	Vehicle Entry	4
preview_fee() / calculate_fee()	Exit & Fee Calculation	5
checkout_vehicle()	Exit & Fee Calculation	5
checkout_mpesa()	Payment Processing	6.2
mpesa_callback()	Payment Processing	6.2
complete_mpesa_checkout()	Payment Processing	6.2
checkout_status()	Payment Processing	6.2
open_barrier()	Barrier Control	7
login_required() / login() / logout()	Staff Authentication	8

Appendix B — Glossary of Terms

Term	Definition
STK Push	'Sim Toolkit Push' — an M-Pesa prompt sent directly to a phone's SIM menu asking the holder to enter their PIN to authorise a payment.
CheckoutRequestID	A unique identifier Safaricom's Daraja API returns when an STK push is initiated, used to match the later callback to the original request.
Daraja API	Safaricom's developer platform providing M-Pesa integration endpoints, including Lipa Na M-Pesa Online.
FIFO	First-In-First-Out — the ordering discipline used by the available-slot queue.
Session (parking)	One record of a vehicle's stay, from entry to exit, whether or not payment has completed.
Active session	A session with no exit_time yet recorded — the vehicle is still parked.
Attendant	A logged-in staff member permitted to record entries and take payment.

Appendix C — Sample End-to-End Walkthrough

This walkthrough traces a single vehicle through the entire system, from arrival to a paid M-Pesa exit, referencing the pseudocode routines and database columns described earlier in this report.

Scenario

A driver in plate KDA 123B arrives at 9:00 AM and parks for just over one hour before returning to pay by M-Pesa.

Step	Actor	System Action	Reference
1	Attendant	Logs in with the shared staff password.	Section 8
2	Attendant	Submits plate KDA 123B on the Vehicle Arrival form.	Section 4
3	System	allocate_slot() pops the next free slot (e.g. S07) from the queue; a new active session row is inserted.	Section 4, 3.2
4	Driver	Returns at 10:05 AM; attendant opens Exit / Pay and looks up the plate.	Section 5
5	System	preview_fee() computes 65 minutes parked; calculate_fee() returns Kshs. 50.	Section 5
6	Attendant	Enters the driver's phone number and selects Lipa na M-Pesa.	Section 6.2
7	System	initiate_mpesa_checkout() tags the session pending; mpesa.stk_push() requests a prompt from Daraja.	Section 6.2
8	Driver	Enters their M-Pesa PIN on their own phone to approve the Kshs. 50 charge.	Section 6.2
9	Safaricom	Posts the result to /mpesa/callback with ResultCode 0 and a receipt number.	Section 6.2
10	System	complete_mpesa_checkout() frees slot S07, marks the session paid, and stores the M-Pesa receipt.	Section 6.2, 3.2
11	Attendant's browser	The waiting screen's next poll receives status 'paid' and redirects to the receipt.	Section 6.2
12	System	open_barrier() marks the transaction barrier='OPEN'; the receipt shows Kshs. 50 paid via M-Pesa.	Section 7

Throughout this sequence, every lookup — finding the active session for KDA 123B, allocating slot S07, and resolving the callback back to the right plate — is an $O(1)$ hash-map or queue operation, regardless of how many other vehicles are simultaneously parked, in line with the complexity analysis in Section 3.4.